

Projectile - Project Interaction

Description	Keystroke	Function	Note
Projectile - Help, Customization, Activation - Define, Protect Project, VCS - Navigate Projects, Project Dired - Project Buffers - Project Files - Open File with different extension - Search/Replace in Project - Project CTags - Project Build, Shell, Commands - Project Test , References Last updated on: 2026-09-20 See the projectile user manual for more information.	The projectile external package is a great package to manage your project. PEL activates projectile when the pel-use-projectile user option is non-nil: t : projectile is available on request. Activate it explicitly with <f11> <f8> <f8> use-from-start : PEL activates projectile when Emacs starts. PEL binds projectile-command-map to <f8> instead of projectile recommended binding C-c p. - PEL key binding will work on all OS in both terminal and GUI mode. - See the various PEL keyboard configuration pages describing how to setup the terminal and the OS keyboard settings for PEL compatible keyboard setting: terminal settings, Mint 20 Desktop Keys, Rocky Linux 8 Desktop Keys, and Ubuntu 16.04 Desktop Keys.		
Open local help PDF	<f11> <f8> <f1> <f8> <f1>	(pel-help-pdf &optional OPEN-WEB-PAGE)	Open the Projectile local PDF. With C-u , M-- , M- -1 or C- -1 prefix: open the remote GitHub hosted PDF instead. If the pel-flip-help-pdf-arg user-option is set it's the other way around. The key sequence <f8> <f1> is only available when the projectile mode is activated.
Customize PEL support for Projectile	<f11> <f2> P <f8> <f11> <f8> <f2> <f8> <f2>	(pel-cfg-pkg-project-mng &optional OTHER-WINDOW)	Customize PEL Project-Mng support. - If OTHER-WINDOW is non-nil (use C-u), display in other window and open the related group(s) that exist. - The key sequence <f8> <f2> is available when the projectile mode is activated.
Customize Projectile	<f11> <f8> <f3> <f8> <f3>	(pel-customize-projectile)	Open the projectile customization group where you can modify projectiles configuration. The key sequence <f8> <f3> is available when the projectile mode is activated.
Toggle projectile mode	<f11> <f8> <f8>	(projectile-mode &optional ARG)	Toggle projectile-mode, a minor mode to assist project management and navigation. PEL activates the projectile-mode when Emacs starts if the pel-use-projectile user option is set to use-from-start. If instead pel-use-projectile is set to t, then you must use this command to activate it.
Projectile Commander Invoke other Projectile commands with single letters Commands are also available via <f8> key sequences. See below. There are some difference between the keys here and the <f8> keys.	<f8> m	(projectile-commander)	Execute a Projectile command with a single letter. - The user is prompted for a single character indicating the action to invoke. - Type '?' for a description of available actions and their keys from this command.
	- You can define new <i>methods</i> by writing Emacs Lisp code which uses the def-projectile-commander-method' to assign specific keys in the map for the action expressed in Emacs Lisp code. - Currently PEL does not provide a mechanism to define this type of code, but you can add this code inside your init.el file after the call to pel-init()).		
Define Projects	Projectile identifies project files with the information it gets from VCS repo definition files (such as .git or .hg) or from the presence and content of a .projectile file in the root directory of a project. Read the Projectile//Docs/Project for more information.		
Configure the project	<f8> C	(projectile-configure-project ARG)	Run project configure command. Normally you'll be prompted for a compilation command, unless variable 'compilation-read-command'. You can force the prompt with a prefix ARG.
Edit/Create a .dir-locals.el file for the project	<f8> E	(projectile-edit-dir-locals)	Edit or create a .dir-locals.el file of the current project. The file is located at the root directory of the project. Command prompts for names of variables and their values and writes the corresponding Emacs Lisp code inside the file.
Project & VCS	Projectile provides the following commands to interoperate with your project VCS. See also: VCS-Git Magit , VCS-Mercurial , VCS-Subversion		
Browse dirty version controlled projects	<f8> v	(projectile-browse-dirty-projects &optional CACHED)	Browse dirty version controlled projects. With a prefix argument, or if CACHED is non-nil, try to use the cached dirty project list.
	Use this to quickly identify your projects that have non-committed files. This may take some time to execute.		
Open the project VCS status buffer	<f8> v	(projectile-vc &optional PROJECT-ROOT)	Open 'vc-dir' at the root of the project. For git projects 'magit-status-internal' is used if available. For hg projects 'monky-status' is used if available.
	If PROJECT-ROOT is given, it is opened instead of the project root directory of the current buffer file. If interactively called with a prefix argument, the user is prompted for a project directory to open.		
Protect project files Toggle project read-only	<f8> ~	(projectile-toggle-project-read-only)	Toggle project read only.
Navigate Projects	Projectile learn about projects when you first visit (or open) a file that is in a repo directory tree or a directory tree that has a .projectile file at its root. See the Projectile Supported Project Types for the list of project types supported and how to add support for more. You can change the active project with the following commands and potentially execute commands on them.		
Switch project	<f8> p	(projectile-switch-project &optional ARG)	Switch to a project previously visited. we have visited before: - Prompt for the name of the project, showing the previously visited projects. - Invokes the command referenced by 'projectile-switch-project-action' on switch. - By default this is projectile-find-file: it prompts for a file within the project - With a prefix ARG invokes 'projectile-commander' instead of 'projectile-switch-project-action.'
Switch to a currently opened project	<f8> q	(projectile-switch-open-project &optional ARG)	Switch to a project we have currently opened. - Invokes the command referenced by 'projectile-switch-project-action' on switch. - With a prefix ARG invokes 'projectile-commander' instead of 'projectile-switch-project-action.'
Expand Speedbar to the entire projectile project directories and files See also: Speedbar	<f8> M-s	(projectile-speedbar-open-current-buffer-in-tree)	With a speedbar already opened, expand it to the entire current projectile project directories and files. To use this command you must first activate projectile. The command fails if issued when point lies inside the speedbar window.
	Requires the projectile-speedbar external project. PEL activates it when the pel-use-projectile-speedbar user-option is set to t.		
Project Dired See also: Dired	Projectile provides the following commands to open a Dired (Directory Editor) buffer for one of the directories related to the current project. - The first 3 commands, bound to a key sequence that end with a D open dired at the root and do not prompt. The other commands prompt for one of the project's directories. - When prompting for a directory name, the list of available directory names is restricted to the directories that are part of the current project.		
Open Dired for the project's root directory.	<f8> D	(projectile-dired)	Open 'dired' at the root of the project. Does not prompt, opens Dired on the project's root directory.
Open Dired for the project's root directory, in other window	<f8> 4 D	(projectile-dired-other-window)	Open 'dired' at the root of the project in another window. Does not prompt, opens Dired on the project's root directory.
Open Dired for the project's root directory, in other frame	<f8> 5 D	(projectile-dired-other-frame)	Open 'dired' at the root of the project in another frame. Does not prompt, opens Dired on the project's root directory.
Open Dired for project directory	<f8> d	(projectile-find-dir &optional INVALIDATE-CACHE)	Jump to a project's directory using completion. With a prefix arg INVALIDATE-CACHE invalidates the cache first.
Open Dired for project directory in other window	<f8> 4 d	(projectile-find-dir-other-window &optional INVALIDATE-CACHE)	Jump to a project's directory in other window using completion. With a prefix arg INVALIDATE-CACHE invalidates the cache first.
Open Dired for project directory in other frame	<f8> 5 d	(projectile-find-dir-other-frame &optional INVALIDATE-CACHE)	Jump to a project's directory in other frame using completion. With a prefix arg INVALIDATE-CACHE invalidates the cache first.

Description	Keystroke	Function	Note
Project Buffers See also: ↗ Buffers	Projectile provides the following commands to manage buffers related to a project. - projectile-buffer opens an Buffer listing only the files in the currently active project, nothing else. - Then shown is a set of commands open a buffer that is part of a project, in the current window, another window with and without selecting it, or into another frame. - When prompting for a buffer name, the lists of available buffers is restricted to the buffers that are part of the current project (as opposed to all buffers currently opened in Emacs). - The last 2 commands listed are: - save all buffers related to the project, - kill all buffers related to the project.		
List project's buffers	<f8> I	(projectile-ibuffer PROMPT-FOR-PROJECT)	Open an IBuffer window showing all buffers in the current project (exclude all others). Let user choose another project when PROMPT-FOR-PROJECT is supplied.
Switch to previous project buffer	<f8> <left>	(projectile-previous-project-buffer)	In selected window switch to the previous project buffer. If the current buffer does not belong to a project, call ‘previous-buffer’.
Switch to next project buffer	<f8> <right>	(projectile-next-project-buffer)	In selected window switch to the next project buffer. If the current buffer does not belong to a project, call ‘next-buffer’.
Switch to a project buffer	<f8> b	(projectile-switch-to-buffer)	Switch to a specific project buffer: prompt for the buffer name.
Switch to a project buffer in other window	<f8> 4 b	(projectile-switch-to-buffer-other-window)	Switch to a specific project buffer and show it in another window.
Open a project buffer in other window with our selecting it.	<f8> 4 C-o	(projectile-display-buffer)	Display a project buffer in another window without selecting it.
Switch to a project buffer in other frame	<f8> 5 b	(projectile-switch-to-buffer-other-frame)	Switch to a project buffer and show it in another frame.
Save all project buffers	<f8> S	(projectile-save-project-buffers)	Save all project buffers.
Kill all project's buffers	<f8> k	(projectile-kill-buffers)	Kill project buffers. Prompts for confirmation before killing the buffers.
Switch to most recently selected buffer	<f8> 1 <f8> ESC	(projectile-project-buffers-other-buffer)	Switch to the most recently selected buffer project buffer. Only buffers not visible in windows are returned.
		 Projectile key map uses the Esc key for this. This is an unfortunate choice because it prevents adding Function or cursor keys to the map since these keys are implemented as ANSI escape sequence codes in terminal mode. PEL removes that key binding and replaces it with 1 (i.e. the number ‘ one ’) which is physically located close to the Esc key on several keyboards .	
Project Files See also: ↗ File mngt	Projectile provides the following commands to open a file in any of the know projects and commands that open a file that belongs to the current project. When prompting for a file name, the list of available file names is restricted to the files that are part of the current project.		
Find (open) a file in any of the known projects	<f8> F	(projectile-find-file-in-known-projects)	Jump to a file in any of the known projects.  This may take a long time with large number of projects. CPU load may go high.
Find (open) project file	<f8> f	(projectile-find-file &optional INVALIDATE-CACHE)	Jump to a project's file using completion. With a prefix arg INVALIDATE-CACHE invalidates the cache first.
Find (open) project file in other window	<f8> 4 f	(projectile-find-file-other-window &optional INVALIDATE-CACHE)	Jump to a project's file using completion and show it in another window. With a prefix arg INVALIDATE-CACHE invalidates the cache first.
Find (open) project file in other frame	<f8> 5 f	(projectile-find-file-other-frame &optional INVALIDATE-CACHE)	Jump to a project's file using completion and show it in another frame. With a prefix arg INVALIDATE-CACHE invalidates the cache first.
Find (open) project file identified at point	<f8> g	(projectile-find-file-dwim &optional INVALIDATE-CACHE)	Jump to a project's files using completion based on context. With a prefix arg INVALIDATE-CACHE invalidates the cache first.
	- If point is on a filename, Projectile first tries to search for that file in project: - If it finds just a file, it switches to that file instantly. This works even if the filename is incomplete, but there's only a single file in the current project that matches the filename at point. For example, if there's only a single file named "projectile/projectile.el" but the current filename is "projectile/proj" (incomplete), ‘projectile-find-file-dwim’ still switches to "projectile/projectile.el" immediately because this is the only filename that matches. - If it finds a list of files, the list is displayed for selecting. A list of files is displayed when a filename appears more than one in the project or the filename at point is a prefix of more than two files in a project. For example, if ‘projectile-find-file-dwim’ is executed on a filepath like "projectile/" , it lists the content of that directory. If it is executed on a partial filename like "projectile/a", a list of files with character ‘a’ in that directory is presented. - If it finds nothing, display a list of all files in project for selecting.		
Find (open) project file identified at point in other window	<f8> 4 g	(projectile-find-file-dwim-other-window &optional INVALIDATE-CACHE)	Jump to a project's files using completion based on context in other window. Same selection logic as for the command described above.
Find (open) project file identified at point in other frame	<f8> 5 g	(projectile-find-file-dwim-other-frame &optional INVALIDATE-CACHE)	Jump to a project's files using completion based on context in other frame. Same selection logic as for the command described above.
Search file in current projectile project with fzf & open selected one. See ↗ File mngt	<f11> f <f8> <f8> M-z	(fzf-projectile &optional WITH-PREVIEW)  Requires the fzf.el external package  activated by pel-use-fzf	Search all files in current projectile project with fzf and open selected file. With C-u show file preview. See fzf manual , fzf search syntax .
Find (open) project file with different extension	Projectile provides the following commands to open a project file that has the same name as the current file but has a different extension. Several programming languages use several file extensions, such as C for .c and .h files, C++ with .cpp, .hpp .hxx, Erlang with .erl, .hrl, etc...And if you have documentation files with the same names that will work too. The files do not have to be located inside the same directory. Example: For a C project you could place all the C header files describing the public interface to your package inside the top directory, and the implementation files inside a code sub-directory (along with private header files). Let's say you have project foo with foo/foo_base.h, foo/code/foo_base.c. If you have foo_base.c opened in a window, use one of these commands to quickly open foo.h.		
Find (open) file with same name but different extension	<f8> a	(projectile-find-other-file &optional FLEX-MATCHING)	Switch between files with the same name but different extensions. - With FLEX-MATCHING, match any file that contains the base name of current file. - Other file extensions can be customized with the variable ‘projectile-other-file-alist’.
Find (open) file with same name but different extension in other window	<f8> 4 a	(projectile-find-other-file-other-window &optional FLEX-MATCHING)	Switch between files with the same name but different extensions in other window. - With FLEX-MATCHING, match any file that contains the base name of current file. - Other file extensions can be customized with the variable ‘projectile-other-file-alist’.
Find (open) file with same name but different extension in other frame	<f8> 5 a	(projectile-find-other-file-other-frame &optional FLEX-MATCHING)	Switch between files with the same name but different extensions in other frame. - With FLEX-MATCHING, match any file that contains the base name of current file. - Other file extensions can be customized with the variable ‘projectile-other-file-alist’.
Open recently opened project file	<f8> e	(projectile-recentf)	Show a list of recently visited files in a project. For this, recentf must be enabled.  PEL enables it when pel-use-recentf is set to t .
Remove current project files	<f8> i	(projectile-invalidate-cache PROMPT)	Remove the current project's files from ‘projectile-projects-cache’. With a prefix argument PROMPT prompts for the name of the project whose cache to invalidate.
Open a file in a project directory.	<f8> 1	(projectile-find-file-in-directory &optional DIRECTORY)	Jump to a file in a (maybe regular) DIRECTORY. First select the directory at the first prompt. Then select the file in the second prompt.
Add currently visited file to the projectile cache.	<f8> z	(projectile-cache-current-file)	Add the currently visited file to the cache.

Description	Keystroke	Function	Note
Search in Project See: ↗ Search/Replace	- Searching in project buffers: projectile provides the multi-occur for project buffers, shown non the first row. - Searching in project files: projectile provides the following recursive-grep like search tools, they are listed starting on the second row. - The first one searches inside buffers, not in files. That may be useful when looking for unsaved buffers or for special buffers. - The last 2 require external packages and external command line utilities that must have been installed separately: ripgrep and ag. - The ripgrep and ag searches are faster than the standard grep search. To navigate through search results use: - move to next occurrence: C-x ` or M-g n or M-g M-n - move to previous occurrence: M-g p or M-g M-p		
Search for occurrence of text in project buffers	<f8> o	(projectile-multi-occur &optional NLINES)	Do a ‘multi-occur’ in the project’s buffers . With a prefix argument, show NLINES of context.
Search in project files with recursive grep	<f8> s g	(projectile-grep &optional REGEXP ARG)	Perform rgrep in the project. - With a prefix ARG asks for files (globbing-aware) which to grep in. - With prefix ARG of ‘-’ (such as ‘M--’), default the files (without prompt), to ‘projectile-grep-default-files’. - With REGEXP given, don’t query the user for a regexp.
Search in project files with ripgrep • Rust (ripgrep) regexp syntax	<f8> s r	(projectile-ripgrep SEARCH-TERM &optional ARG)	Run a ripgrep search with ‘SEARCH-TERM’ at current project root. With optional prefix argument ARG SEARCH-TERM is interpreted as a regular expression. 📦 Requires the ripgrep.el external packages as well as the ripgrep command line utility. 🔧 To make it work you must set pel-use-ripgrep to t. ⚠️ Note that the ripgrep command line utility must be installed manually.
Search in project files with ag • PCRE regex syntax	<f8> s s	(projectile-ag SEARCH-TERM &optional ARG)	Run an ag search with SEARCH-TERM in the project. With optional prefix argument ARG SEARCH-TERM is interpreted as a regular expression. 📦 Requires the ag.el external packages as well as the ag command line utility. 🔧 To make it work you must set pel-use-ag to t. ⚠️ Note that the ag command line utility must be installed manually.
Replace in Project	Text replacement inside all project files.		
Replace test in project files	<f8> r	(projectile-replace &optional ARG)	Replace literal string in project using non-regexp ‘tags-query-replace’. With a prefix argument ARG prompts you for a directory on which to run the replacement.
Project CTags  - generate TAGS file - jump to definition See ↗ Xref for commands using the generated tags.	Projectile supports the ctags file cross referencing utility by being able to regenerate the tags/TAGS file explicitly or periodically via a timer. - The tool used is controlled by the projectile-tags-backend user option. The default is set to auto. - The shell command projectile uses is identified in the projectile-tags-command user option. The default is set to <code>ctags -Re -f "%s" %s “%s”</code>. - This is the command line for the now defunct exuberant-ctags. That project is no longer maintained. You can use Universal Ctags instead, it is also called ctags. - That is unfortunate because that conflicts with GNU ctags and Emacs ctags. - One way to solve the problem is to create links to the Universal Ctags executable named uctags and do the same for readtags (ureadtags), then update the value of projectile-tags-command to <code>uctags -Re -f "%s" %s “%s”</code>. - Another solution is to rename other of those programs and leave ctags for the Universal ctags. - You can also set projectile-tags-backend to use gtags: the GNU global tagging system. - If you want projectile to automatically regenerate the tags periodically, customize the projectile-enable-idle-timer user option. - PEL provides the <f8> <f1> key binding to access the projectile customization group quickly.		
Regenerate project’s Tags file	<f8> R	(projectile-regenerate-tags)	Regenerate the project’s [elg]tags.
Jump to project’s (tag) symbol definition	<f8> j	(projectile-find-tag)	Find tag in project. - Prompt for a target (like the name of a function) and then open the file at that location. Tag defaults to symbol at point. - To go back where point was, use M- ,
Run Projects commands			
Run project compilation command	<f8> c	(projectile-compile-project ARG)	Run project compilation command. - Normally you’ll be prompted for a compilation command, unless variable ‘compilation-read-command’. You can force the prompt with a prefix ARG. The result of the command is shown in a *compilation* buffer that supports the following keys: TAB : move to next error backtab : move to previous error C-o : move to the source where the error is reported o : open a visible link in a compilation-mode g : re-run the command
Execute project’s “run” command	<f8> u	(projectile-run-project ARG)	Run project run command. - Normally you’ll be prompted for a compilation command, unless variable ‘compilation-read-command’. You can force the prompt with a prefix ARG. - The result is also shown in a *compilation* buffer, with the same key commands available.
Run Shell Commands in Project’s root	The following projectile commands execute shell commands in the project’s root directory.		
Run shell command in project root	<f8> !	(projectile-run-shell-command-in-root)	Invoke ‘shell-command’ in the project’s root. Display results in a “Shell Command Output” buffer.
Run async command in project root	<f8> &	(projectile-run-async-shell-command-in-root)	Invoke ‘async-shell-command’ in the project’s root. Display results in a “Shell Command Output” buffer.
Specialized Shells	Open a specialized shell related to the current project. The buffer name reflect the name of the project’s root directory. See the ↗ Shells table for more information on these shells.		
eshell	<f8> x e	(projectile-run-eshell ARG)	Invoke ‘eshell’ in the project’s root. - Switch to the project specific eshell buffer if it already exists. - Use a prefix argument ARG to indicate creation of a new process instead.
gdb	<f8> x g	(projectile-run-gdb)	Invoke ‘gdb’ in the project’s root.
ielm	<f8> x i	(projectile-run-ielm ARG)	Invoke ‘ielm’ in the project’s root. - Switch to the project specific ielm buffer if it already exists. - Use a prefix argument ARG to indicate creation of a new process instead.
shell	<f8> x s	(projectile-run-shell ARG)	Invoke ‘shell’ in the project’s root. - Switch to the project specific shell buffer if it already exists. - Use a prefix argument ARG to indicate creation of a new process instead.
term	<f8> x t	(projectile-run-term ARG)	Invoke ‘term’ in the project’s root. - Switch to the project specific term buffer if it already exists. - Use a prefix argument ARG to indicate creation of a new process instead.
vterm	<f8> x v	(projectile-run-vterm &optional ARG)	Invoke ‘vterm’ in the project’s root. - Switch to the project specific term buffer if it already exists. - Use a prefix argument ARG to indicate creation of a new process instead.

Description	Keystroke	Function	Note
Project's Tests			
Toggle between and implementation file and its test file	<f8> t	(projectile-toggle-between-implementation-and-test)	Toggle between an implementation file and its test file.
Open matching test file in other window	<f8> 4 t	(projectile-find-implementation-or-test-other-window)	Open matching implementation or test file in other window.
Open matching implementation file in other frame	<f8> 5 t	(projectile-find-implementation-or-test-other-frame)	Open matching implementation or test file in other frame.
Run project's test	<f8> P	(projectile-test-project ARG)	Run project test command. Normally you'll be prompted for a compilation command, unless variable 'compilation-read-command'. You can force the prompt with a prefix ARG.
Jump to project test file	<f8> T	(projectile-find-test-file &optional INVALIDATE-CACHE)	Jump to a project's test file using completion. With a prefix arg INVALIDATE-CACHE invalidates the cache first.

Projectile – References

Topic & Link		Note
Projectile		
Projectile @ Github		
Projectile User Manual		
Recommended tools		When using Projectile it is best to also install these command line tools to get the best performance.
Find Replacements	fd	fd is a file finder hat performs much faster than find .
Grep Replacements	ripgrep : rg	ripgrep is a fast grep. The command line name is rg . Uses Rust regular expressions . Read the author's blog on ripgrep for more info. Note: ripgrep and ag both support .ignore file to identify the file patterns to ignore in its search. See this blog entry for history.
	ag	ag is another fast grep. Uses PCRE regular expressions.
Ctags Cross Reference tools	Universal Ctags	A ctag program, successor of the now defunct Exuberant CTags . See Universal CTags Github repository and the Universal CTags Hacking Guide for more info.
	Gnu Global	